Data Science - Unit I & II Answers

- Data Science – Unit I & Unit II: Question Bank Answers (10 Marks Each)
- UNIT I – INTRODUCTION TO DATA SCIENCE
  - Q1. Define Data Science. Explain its scope and importance in various fields. (10M)
  - Q2. Explain the history and evolution of Data Science. (10M)
  - Q3. Explain the major applications of Data Science. (10M)
  - Q4. Explain the Data Science Lifecycle with a neat diagram and describe each stage. (10M)
  - Q5. Compare arrays, lists, matrices, and Data Frames with suitable examples. (10M)
  - Q6. Explain the important tools and technologies used in Data Science. (10M)
  - Q7. Write a Python program to import a dataset using Pandas and perform basic exploration using head(), tail(), shape, info(), and describe(). (10M)
- UNIT II – DATA COLLECTION AND PRE-PROCESSING
  - Q8. Explain different methods of data collection with suitable examples. (10M)
  - Q9. What is data pre-processing? Explain the major steps involved in data pre-processing. (10M)
  - Q10. Explain data cleaning techniques with examples. (10M)
  - Q11. Explain different techniques for handling missing values with an example. (10M)
  - Q12. Explain normalization and standardization. Compare them with suitable examples and formulas. (10M)
  - Q13. What is feature engineering? Explain different feature engineering techniques with examples. (10M)
  - Q14. Explain feature scaling with suitable example. (10M)
  - Q15. Explain data integration and data aggregation with suitable example. (10M)
  - Q16. For a given dataset, calculate the mean, median, mode, and standard deviation. Explain what each statistic represents. (10M)
  - Q17. Explain how to identify and handle outliers in a dataset using suitable techniques such as filtering and discretization. (10M)

Data Science – Unit I & Unit II: Question Bank Answers (10 Marks Each)

UNIT I – INTRODUCTION TO DATA SCIENCE

Q1. Define Data Science. Explain its scope and importance in various fields. (10M)

Definition: Data Science is an interdisciplinary field that uses scientific methods, algorithms, processes, and systems to extract knowledge and actionable insights from structured and unstructured data. It combines domain knowledge, mathematics/statistics, and computer science (programming) to analyze data and support decision-making.

A commonly used definition: *"Data Science is the art of uncovering the insights and trends hidden behind data."*

Components of Data Science (the three pillars): 1. **Mathematics/Statistics** – probability, inferential statistics, linear algebra. 2. **Computer Science/Programming** – Python, R, SQL, databases, big data tools. 3. **Domain Knowledge/Business Understanding** – understanding the problem context to interpret results meaningfully.

Scope of Data Science: - Predictive analytics and forecasting - Descriptive and diagnostic analytics - Prescriptive analytics (recommendation systems) - Machine Learning and Artificial Intelligence model building - Big Data processing and real-time analytics - Business Intelligence and dashboarding - Automation of decision-making processes

Importance in Various Fields:

Field	Application
Healthcare	Disease prediction, drug discovery, medical imaging diagnosis
Finance/Banking	Fraud detection, credit risk scoring, algorithmic trading
E-commerce	Recommendation engines, customer segmentation, demand forecasting
Manufacturing	Predictive maintenance, quality control, supply chain optimization
Government	Policy planning, crime prediction, census analysis

Transportation	Route optimization, self-driving cars, traffic prediction
Education	Personalized learning, student performance prediction
Sports	Player performance analytics, injury prediction

Data Science is important because organizations today generate massive volumes of data, and converting this raw data into meaningful insight gives a **competitive advantage**, improves **efficiency**, reduces **costs**, and enables **data-driven decision-making** rather than intuition-based decisions.

Q2. Explain the history and evolution of Data Science. (10M)

Data Science evolved gradually from statistics and computer science into its own discipline:

1960s–1970s: Foundations - The term “Data Science” was loosely used by Peter Naur (1974) as a substitute for “computer science.” - Statisticians used exploratory data analysis (John Tukey, 1962) — considered a precursor.

1980s–1990s: Knowledge Discovery in Databases (KDD) - Growth of databases and the emergence of “Knowledge Discovery in Databases” (KDD) and Data Mining as fields. - 1996: The term “Data Science” was formally used at the International Federation of Classification Societies conference.

2000s: Formal Recognition - William S. Cleveland (2001) published “Data Science: An Action Plan,” proposing it as an independent discipline extending statistics. - 2008: DJ Patil and Jeff Hammerbacher (LinkedIn and Facebook) coined the modern job title “Data Scientist.”

2010s: Big Data Era - Explosive growth of internet, social media, and IoT led to “Big Data” (Volume, Velocity, Variety). - Hadoop, Spark, and NoSQL databases emerged to handle large-scale data. - Harvard Business Review (2012) called Data Scientist “The Sexiest Job of the 21st Century.”

2015–Present: AI/ML Integration - Integration of Machine Learning, Deep Learning, and Cloud Computing. - Rise of automated ML (AutoML), MLOps, and large-scale AI models. - Data Science became central to business strategy across nearly every industry.

Evolution Summary: Statistics → Data Mining/KDD → Business Intelligence → Big

Data → Machine Learning → Data Science → AI-driven Data Science (current era)

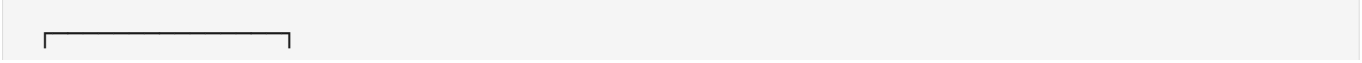
Q3. Explain the major applications of Data Science. (10M)

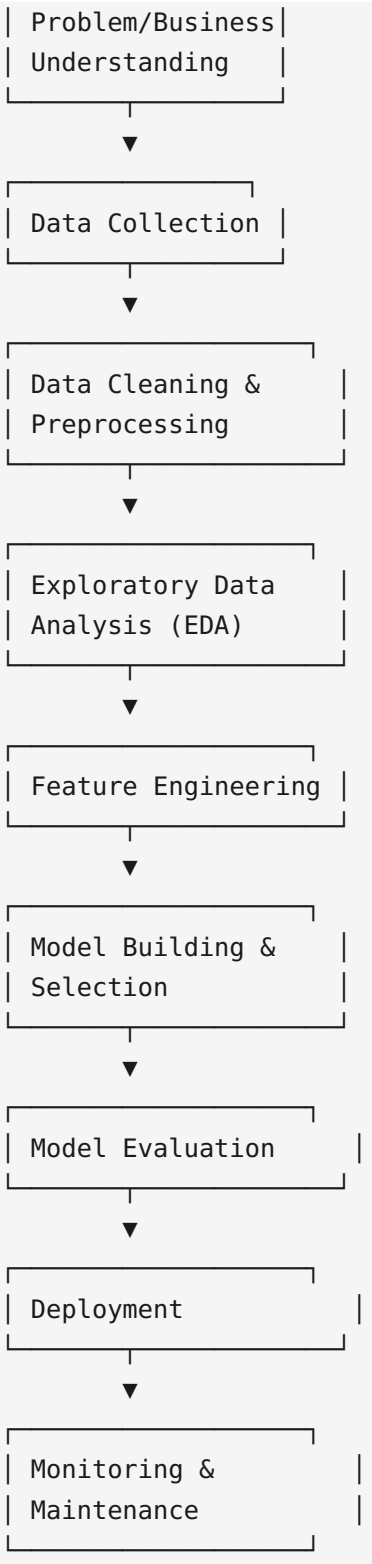
1. **Healthcare** – Disease diagnosis using medical imaging, drug discovery, genomics analysis, predicting epidemics.
2. **Banking & Finance** – Credit scoring, fraud detection, algorithmic trading, risk management.
3. **Retail & E-commerce** – Recommendation systems (Amazon, Flipkart), customer segmentation, dynamic pricing, inventory management.
4. **Transportation** – Uber/Ola route optimization, self-driving vehicles, traffic prediction.
5. **Manufacturing** – Predictive maintenance of machines, quality assurance, supply chain optimization.
6. **Social Media** – Sentiment analysis, targeted advertising, content recommendation (Facebook, Instagram, YouTube).
7. **Search Engines** – Google's search ranking algorithms use data science for relevance ranking.
8. **Airlines and Logistics** – Fare prediction, delay prediction, route planning.
9. **Government & Public Policy** – Crime prediction (predictive policing), census data analysis, election forecasting.
10. **Sports Analytics** – Player performance tracking, strategy optimization (e.g., IPL, cricket analytics).
11. **Agriculture** – Crop yield prediction, soil analysis, precision farming.
12. **Entertainment** – Netflix/Spotify recommendation engines based on user viewing/listening history.
13. **Cybersecurity** – Anomaly detection, intrusion detection systems.
14. **Human Resources** – Employee attrition prediction, resume screening automation.

These applications show that Data Science is not restricted to IT but spans nearly every sector of the economy, enabling smarter, faster, and more accurate decisions.

Q4. Explain the Data Science Lifecycle with a neat diagram and describe each stage. (10M)

Diagram (Data Science Lifecycle):





Stages Explained:

1. **Business/Problem Understanding** – Defining the objective clearly: what business problem needs to be solved (e.g., predicting customer churn).
2. **Data Collection** – Gathering data from various sources: databases, APIs, web scraping, sensors, surveys.

3. **Data Cleaning & Preprocessing** – Handling missing values, removing duplicates, correcting inconsistent data, dealing with outliers.
4. **Exploratory Data Analysis (EDA)** – Understanding data through statistics and visualization (histograms, box plots, correlation matrices) to identify patterns and relationships.
5. **Feature Engineering** – Creating new relevant features, selecting important variables, transforming data (scaling, encoding).
6. **Model Building** – Selecting and training suitable Machine Learning/Statistical algorithms (regression, classification, clustering) on the data.
7. **Model Evaluation** – Testing model performance using metrics like accuracy, precision, recall, RMSE, etc., and validating with test/cross-validation data.
8. **Deployment** – Integrating the model into a production environment/application for real-world use.
9. **Monitoring & Maintenance** – Continuously tracking model performance over time and retraining as data patterns change (model drift).

This lifecycle is often **iterative**, not strictly linear — insights from later stages often require revisiting earlier stages.

Q5. Compare arrays, lists, matrices, and Data Frames with suitable examples. (10M)

Feature	Array	List	Matrix	DataFrame
Definition	Collection of homogeneous elements stored in contiguous memory	Ordered, mutable collection that can hold heterogeneous data types	2-D array with rows and columns, all elements of same type	2-D labeled tabular data structure with rows and columns, columns can have different data types
Library	NumPy (`numpy.array`)	Built-in Python	NumPy (`numpy.matrix` or 2D array)	Pandas (`pandas.DataFrame`)
Data Type	Homogeneous	Heterogeneous	Homogeneous	Heterogeneous (per column)
		1-D (nested for		

Dimensions	1-D, 2-D, n-D	multi-D)	Strictly 2-D	2-D (rows × columns)
Indexing	Integer-based	Integer-based	Row, column integer index	Labeled index (row/column names) or integer
Mutability	Fixed size (in NumPy)	Mutable, dynamic size	Fixed size	Mutable, dynamic
Example	`import numpy as np a = np.array([1,2,3])`	`l = [1, "data", 3.5, True]`	`m = np.array([[1,2],[3,4]])`	`import pandas as pd df = pd.DataFrame({'Name':['A','B'],'Age':[20,21]})`
Use Case	Numerical computations	General-purpose storage	Linear algebra operations	Data analysis, tabular datasets (like Excel/SQL tables)

Example illustration:

```
import numpy as np
import pandas as pd

# List
my_list = [1, "Data", 3.5]

# Array
my_array = np.array([1, 2, 3, 4])

# Matrix
my_matrix = np.array([[1, 2], [3, 4]])

# DataFrame
my_df = pd.DataFrame({'ID':[1,2], 'Name':['John','Amy']})
```

Key takeaway: Lists are flexible but slow for numeric computation; Arrays and Matrices are fast and memory-efficient for numerical operations; DataFrames are best suited for real-world structured/tabular data analysis with labeled rows/columns.

Q6. Explain the important tools and technologies used in Data Science. (10M)

1. Programming Languages - **Python** – Most popular; libraries like Pandas,

NumPy, Scikit-learn, TensorFlow. - **R** – Strong in statistical analysis and visualization. - **SQL** – For querying and managing relational databases.

2. Data Manipulation & Analysis Libraries - **Pandas** – Data manipulation and analysis using DataFrames. - **NumPy** – Numerical computing with arrays.

3. Data Visualization Tools - **Matplotlib, Seaborn** (Python) – Static plots and charts. - **Tableau, Power BI** – Interactive dashboards for business intelligence. - **Plotly** – Interactive visualizations.

4. Machine Learning & AI Frameworks - **Scikit-learn** – Traditional ML algorithms. - **TensorFlow, PyTorch, Keras** – Deep learning frameworks.

5. Big Data Technologies - **Hadoop** – Distributed storage and processing (HDFS, MapReduce). - **Apache Spark** – Fast, in-memory big data processing. - **Hive, Pig** – Query and scripting tools on Hadoop.

6. Databases - **Relational**: MySQL, PostgreSQL, Oracle. - **NoSQL**: MongoDB, Cassandra, Redis.

7. Cloud Platforms - **AWS, Google Cloud Platform (GCP), Microsoft Azure** – Provide scalable storage, computing, and ML services (SageMaker, BigQuery, Azure ML).

8. IDEs & Notebooks - **Jupyter Notebook, Google Colab, Spyder, VS Code** – For interactive coding and analysis.

9. Version Control - **Git/GitHub** – For code collaboration and version tracking.

10. Other Tools - **Excel** – Basic data analysis. - **Apache Kafka** – Real-time data streaming. - **Docker/Kubernetes** – Deployment and containerization of models.

These tools together form the technology stack that supports every stage of the data science lifecycle — from data collection to deployment.

Q7. Write a Python program to import a dataset using Pandas and perform basic exploration using head(), tail(), shape, info(), and describe(). (10M)

```
# Import necessary library
import pandas as pd

# Import the dataset (CSV file example)
```

```
        df = pd.read_csv("dataset.csv")

        # 1. Display first 5 rows
        print("First 5 rows of the dataset:")
        print(df.head())

        # 2. Display last 5 rows
        print("\nLast 5 rows of the dataset:")
        print(df.tail())

        # 3. Display shape (rows, columns)
        print("\nShape of the dataset (rows, columns):")
        print(df.shape)

        # 4. Display concise summary - datatypes, non-null counts
        print("\nInfo about the dataset:")
        print(df.info())

        # 5. Display statistical summary of numeric columns
        print("\nStatistical Description of the dataset:")
        print(df.describe())
```

Explanation of functions: - **`head(n)`** – Displays the first `n` rows (default 5) of the dataset; used to get a quick look at the data. - **`tail(n)`** – Displays the last `n` rows (default 5); useful to check the end of the dataset. - **`shape`** – Returns a tuple `(rows, columns)` giving the dimensions of the dataset. - **`info()`** – Provides a summary including column names, data types, non-null value counts, and memory usage — useful to detect missing values and datatype issues. - **`describe()`** – Generates descriptive statistics (count, mean, std, min, 25%, 50%, 75%, max) for numerical columns, helping understand data distribution.

Sample Output (for a typical dataset):

```
Shape of the dataset (rows, columns):
(150, 5)

Info about the dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   SepalLength   150 non-null    float64
 1   SepalWidth    150 non-null    float64
 ...
```

This basic exploration is the first step of Exploratory Data Analysis (EDA), helping identify data types, missing values, and overall structure before further preprocessing.

UNIT II – DATA COLLECTION AND PRE-PROCESSING

Q8. Explain different methods of data collection with suitable examples. (10M)

Data collection methods can broadly be classified into **Primary** and **Secondary** methods:

A. Primary Data Collection Methods (data collected first-hand) 1. **Surveys/Questionnaires** – Structured forms to gather responses. *Example: Customer satisfaction survey.* 2. **Interviews** – Direct, structured/unstructured conversations. *Example: HR conducting exit interviews.* 3. **Observation Method** – Recording behavior/events without direct interaction. *Example: Observing customer footfall in a store.* 4. **Experiments** – Controlled trials to test hypotheses. *Example: A/B testing website design.* 5. **Sensors/IoT Devices** – Automatic data capture. *Example: Temperature sensors in weather stations.*

B. Secondary Data Collection Methods (data collected by others, reused) 1. **Web Scraping** – Extracting data from websites programmatically using tools like BeautifulSoup, Scrapy. *Example: Scraping product prices from e-commerce sites.* 2. **APIs (Application Programming Interfaces)** – Fetching data from platforms. *Example: Twitter API for tweets, Google Maps API.* 3. **Public/Government Databases** – *Example: Census data, data.gov.in, WHO datasets.* 4. **Organizational Databases/Records** – Company's internal transactional/sales databases (SQL databases, ERP/CRM systems). 5. **Social Media Data** – Data from platforms like Facebook, Instagram for sentiment analysis. 6. **Third-party Data Providers** – Purchased datasets from market research firms (e.g., Nielsen).

Comparison Table:

Method	Cost	Accuracy	Time Required
Surveys	Medium	Medium	Medium
Web Scraping	Low	High (if site reliable)	Fast

APIs	Low-Medium	High	Fast
Government Databases	Low	High	Fast
Experiments	High	Very High	Slow

Choosing the right method depends on factors such as **cost, time, accuracy required, and the nature of the problem**.

Q9. What is data pre-processing? Explain the major steps involved in data pre-processing. (10M)

Definition: Data Pre-processing is the process of transforming raw, messy, incomplete, or inconsistent data into a clean, structured, and usable format suitable for analysis or model building. Since real-world data is often "dirty" (missing values, noise, duplicates), preprocessing is a critical step that directly affects model accuracy — following the principle "**Garbage In, Garbage Out**."

Major Steps in Data Pre-processing:

1. **Data Cleaning**
   - Handling missing values (imputation/removal)
   - Removing duplicate records
   - Correcting inconsistent/incorrect data (e.g., "Male"/"M" standardized)
   - Removing/handling noise and outliers
2. **Data Integration**
   - Combining data from multiple sources (databases, files, APIs) into a single, coherent dataset.
   - Resolving schema conflicts (e.g., different column names for the same attribute).
3. **Data Transformation**
   - **Normalization/Standardization** – Scaling numeric values to a common range.
   - **Encoding categorical variables** – Converting text categories to numbers (Label Encoding, One-Hot Encoding).
   - **Aggregation** – Summarizing data (e.g., daily sales → monthly sales).
4. **Data Reduction**
   - **Dimensionality Reduction** – Reducing number of features (PCA).
   - **Feature Selection** – Selecting only relevant/important features.
   - **Sampling** – Reducing volume of data while preserving patterns.

5. **Data Discretization**
   - Converting continuous data into discrete bins/categories (e.g., Age → Age Groups: 0-18, 19-35, 36-60).
6. **Feature Engineering**
   - Creating new meaningful features from existing data to improve model performance.

Importance: Proper pre-processing improves data quality, reduces computational cost, removes bias, and significantly increases the accuracy and reliability of the final Data Science/ML model.

Q10. Explain data cleaning techniques with examples. (10M)

Data cleaning is the process of detecting and correcting/removing corrupt, inaccurate, incomplete, or irrelevant data.

Major Data Cleaning Techniques:

1. **Handling Missing Data**
   - Removing rows/columns with missing values, or imputing them using mean/median/mode.
   - *Example:* If "Age" column has missing values, replace with the mean age.
2. **Removing Duplicate Records**
   - Identifying and removing duplicate rows to avoid biased results.
   - *Example:* `df.drop_duplicates()` in Pandas.
3. **Correcting Structural Errors**
   - Fixing typos, inconsistent capitalization/labeling.
   - *Example:* "Male", "male", "M" → standardized to "Male".
4. **Handling Outliers**
   - Detecting and treating extreme values using Z-score, IQR method, or visualization (box plots).
   - *Example:* An employee's age recorded as "250" is clearly an outlier/error.
5. **Data Type Conversion**
   - Ensuring columns have correct data types (e.g., converting "20" stored as text to integer).
6. **Handling Inconsistent Data**
   - Resolving conflicting values across sources.
   - *Example:* Date formats like "12-01-2024" vs "2024/01/12" standardized to one format.
7. **Removing Irrelevant Data**

- Dropping columns/features not useful for analysis (e.g., a “Customer Comments” free-text column not needed for numeric prediction).

3. **Noise Reduction**
   - Using techniques like **binning**, **regression**, or **clustering** to smooth noisy data.

Example (Python):

```python
import pandas as pd
df = pd.read_csv("data.csv")

# Remove duplicates
df = df.drop_duplicates()

# Fill missing values with mean
df['Age'].fillna(df['Age'].mean(), inplace=True)

# Standardize text
df['Gender'] = df['Gender'].str.strip().str.capitalize()

# Convert data type
df['Salary'] = df['Salary'].astype(float)
```

Effective data cleaning ensures data quality, consistency, and reliability, which is essential before performing any analysis or modeling.

Q11. Explain different techniques for handling missing values with an example. (10M)

Missing values occur due to data entry errors, non-response in surveys, sensor failures, etc. Handling them properly is crucial.

Techniques for Handling Missing Values:

1. Deletion Methods - **Listwise Deletion (Row deletion):** Remove entire rows with missing values. `python df.dropna(inplace=True)` *Used when missing data is small in proportion (<5%).* - **Column Deletion:** Remove a column if it has too many missing values (e.g., >50% missing). `python df.drop('column_name', axis=1, inplace=True)`

2. Imputation Methods - **Mean Imputation:** Replace missing numeric values with the mean of the column. `python df['Age'].fillna(df['Age'].mean(), inplace=True)` *Example: Ages = [25, 30, NaN, 35] → mean = 30 → NaN replaced with 30.*

- **Median Imputation:** Used when data has outliers (median is robust to outliers).

```
df['Salary'].fillna(df['Salary'].median(), inplace=True)
```

- **Mode Imputation:** Used for categorical data — replace with the most frequent value.

```
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
```

3. Forward Fill / Backward Fill (common in time-series data)

```
df.fillna(method='ffill', inplace=True)  # Forward fill
df.fillna(method='bfill', inplace=True)  # Backward fill
```

4. Predictive Imputation - Using regression, KNN, or ML models to predict and fill missing values based on other features. `python from sklearn.impute import KNNImputer imputer = KNNImputer(n_neighbors=3) df_filled = imputer.fit_transform(df)`

5. Using a Constant/Placeholder Value - Replace missing values with a fixed value like 0, “Unknown”, or “N/A”. `python df['City'].fillna('Unknown', inplace=True)`

Choosing a technique depends on the amount of missing data, whether it’s numerical/categorical, and whether the missingness is random (MCAR/MAR) or systematic (MNAR).

Q12. Explain normalization and standardization. Compare them with suitable examples and formulas. (10M)

Both are **feature scaling** techniques used to bring numeric features to a common scale, which is essential for algorithms sensitive to magnitude (e.g., KNN, Gradient Descent, Neural Networks).

Normalization (Min-Max Scaling) Rescales data to a fixed range, usually **[0, 1]**.

Formula:

```
X_normalized = (X - X_min) / (X_max - X_min)
```

Example: Data = [10, 20, 30, 40, 50]; X_min=10, X_max=50 For X=30: (30-10)/(50-10) = 20/40 = 0.5

Standardization (Z-score Scaling) Rescales data to have **mean = 0** and **standard deviation = 1**.

Formula:

```
X_standardized = (X - μ) / σ
```

where μ = mean, σ = standard deviation.

Example: Data = [10, 20, 30, 40, 50]; μ=30, σ≈14.14 For X=30: (30-30)/14.14 = 0

Comparison Table:

Aspect	Normalization	Standardization
Range	Bounded, typically [0,1]	Not bounded; mean=0, SD=1
Formula	(X-Xmin)/(Xmax-Xmin)	(X-μ)/σ
Sensitive to outliers	Yes, highly sensitive	Less sensitive
Best used when	Data doesn't follow Gaussian distribution; algorithms like KNN, Neural Networks	Data follows Gaussian (normal) distribution; algorithms like Logistic Regression, SVM, PCA
Also known as	Min-Max Scaling	Z-score Scaling

Python Example:

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

data = [[10],[20],[30],[40],[50]]

# Normalization
norm_scaler = MinMaxScaler()
normalized_data = norm_scaler.fit_transform(data)

# Standardization
std_scaler = StandardScaler()
standardized_data = std_scaler.fit_transform(data)
```

Conclusion: Normalization is preferred when the algorithm needs bounded values, while standardization is preferred when data is normally distributed or contains outliers.

Q13. What is feature engineering? Explain different feature engineering techniques with examples. (10M)

Definition: Feature Engineering is the process of using domain knowledge to create, transform, or select variables (features) from raw data that make machine learning algorithms work more effectively, improving model accuracy and performance.

Feature Engineering Techniques:

1. **Feature Creation**
   - Deriving new features from existing ones.
   - *Example:* From "Date of Birth," create a new feature "Age."
2. **Feature Transformation**
   - Applying mathematical functions (log, square root) to reduce skewness.
   - *Example:* `log(Income)` to normalize a right-skewed income distribution.
3. **Encoding Categorical Variables**
   - **Label Encoding:** Assigning numeric labels (e.g., Low=0, Medium=1, High=2).
   - **One-Hot Encoding:** Creating binary columns for each category (e.g., Color_Red, Color_Blue, Color_Green).

```
pd.get_dummies(df['Color'])
```

4. **Feature Scaling**
   - Normalization/Standardization (as explained in Q12) to bring features to a common scale.
5. **Binning/Discretization**
   - Converting continuous variables into categorical bins.
   - *Example:* Age → "Child", "Adult", "Senior".
6. **Feature Extraction**
   - Extracting useful info from complex data types.
   - *Example:* Extracting "Day," "Month," "Year" from a Date column; extracting word counts from text.
7. **Feature Selection**
   - Selecting the most relevant features and removing redundant ones using techniques like Correlation analysis, Chi-square test, or Recursive Feature Elimination (RFE).
8. **Handling Interaction Features**
   - Creating features that capture interaction between two variables.
   - *Example:* "Price per Square Foot" = Price / Area.

9. **Dimensionality Reduction**
    - Using PCA (Principal Component Analysis) to reduce the number of features while retaining most information.

Example:

```
# Creating Age from Date of Birth
df['Age'] = 2026 - pd.to_datetime(df['DOB']).dt.year

# One-hot encoding
df = pd.get_dummies(df, columns=['City'])
```

Effective feature engineering often has more impact on model performance than the choice of algorithm itself.

Q14. Explain feature scaling with suitable example. (10M)

Definition: Feature Scaling is a data preprocessing technique used to standardize the range of independent variables/features so that no feature dominates others due to differences in magnitude/units. It is essential for distance-based algorithms (KNN, K-Means, SVM) and gradient-based algorithms (Neural Networks, Logistic Regression).

Why Feature Scaling is Needed: Consider a dataset with two features: “Salary” (ranging from 20,000 to 100,000) and “Age” (ranging from 20 to 60). Without scaling, “Salary” would dominate distance calculations simply due to its larger magnitude, even though “Age” might be equally or more important.

Common Feature Scaling Techniques:

1. **Min-Max Scaling (Normalization)**

    ```
    X' = (X - Xmin) / (Xmax - Xmin)
    ```

    Scales values between 0 and 1.

2. **Standardization (Z-score Normalization)**

    ```
    X' = (X - μ) / σ
    ```

    Centers data around mean 0 with unit variance.

3. **Robust Scaling** Uses median and IQR (Interquartile Range) — robust to outliers.

```
X' = (X - median) / IQR
```

4. **MaxAbs Scaling** Scales data by dividing by the maximum absolute value, keeping data within [-1, 1].

Example: Dataset: Salary = [20000, 40000, 60000, 80000, 100000]

Using Min-Max Scaling: - Xmin = 20000, Xmax = 100000 - For X = 60000: (60000-20000)/(100000-20000) = 40000/80000 = 0.5

Python Example:

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

data = pd.DataFrame({'Salary':[20000,40000,60000,80000,100000]})
scaler = MinMaxScaler()
data['Salary_scaled'] = scaler.fit_transform(data[['Salary']])
print(data)
```

Conclusion: Feature scaling ensures all features contribute equally to model training, improving convergence speed and accuracy of ML algorithms.

Q15. Explain data integration and data aggregation with suitable example. (10M)

Data Integration: Data Integration is the process of combining data from multiple, heterogeneous sources (databases, files, APIs, spreadsheets) into a single, unified view for analysis.

Challenges in Data Integration: - **Schema Conflicts:** Same attribute named differently across sources (e.g., “Cust_ID” vs “CustomerID”). - **Entity Identification Problem:** Same real-world entity represented differently (e.g., “IBM” vs “International Business Machines”). - **Data Redundancy:** Duplicate records appearing across merged sources. - **Value Conflicts:** Different units/formats (e.g., weight in kg vs lbs).

Example: Merging a “Sales” database from a physical store with an “Online Orders” database from a website to get total company sales.

```
import pandas as pd
store_sales = pd.read_csv("store_sales.csv")
online_sales = pd.read_csv("online_sales.csv")

combined_sales = pd.merge(store_sales, online_sales, on='ProductID', how='outer
```

Data Aggregation: Data Aggregation is the process of summarizing detailed data into a compact form by grouping and applying summary functions (sum, average, count, etc.), often to enable higher-level analysis and reporting.

Example: Aggregating daily sales data into monthly totals.

```
df['Date'] = pd.to_datetime(df['Date'])
monthly_sales = df.groupby(df['Date'].dt.to_period('M'))['Sales'].sum()
print(monthly_sales)
```

Another Example: Aggregating student marks by department:

```
df.groupby('Department')['Marks'].mean()
```

Comparison:

Aspect	Data Integration	Data Aggregation
Purpose	Combine data from multiple sources	Summarize data into compact form
Output	Unified dataset	Summary statistics (grouped)
Example	Merging sales & customer databases	Total sales per region/month

Both techniques are essential in preprocessing to create a consolidated, analysis-ready dataset from raw, distributed, and detailed data.

Q16. For a given dataset, calculate the mean, median, mode, and standard deviation. Explain what each statistic represents. (10M)

Sample Dataset (Marks of 10 students):

```
Data = [45, 55, 60, 60, 65, 70, 72, 75, 80, 90]
```

1. Mean (Average)

```
Mean = Σx / n = (45+55+60+60+65+70+72+75+80+90) / 10
     = 672 / 10 = 67.2
```

Represents: The central/average value of the dataset. It is sensitive to outliers/extreme values.

2. Median (Middle Value) Data sorted: 45,55,60,60,65,70,72,75,80,90 (n=10, even)

```
Median = (5th value + 6th value) / 2 = (65+70)/2 = 67.5
```

Represents: The middle value that separates the higher half from the lower half of the data. It is robust to outliers, making it a better measure of central tendency for skewed data.

3. Mode (Most Frequent Value)

```
Mode = 60 (appears twice, more than any other value)
```

Represents: The most frequently occurring value in the dataset. Useful for categorical/discrete data; a dataset can have no mode, one mode (unimodal), or multiple modes (multimodal).

4. Standard Deviation (Measure of Spread)

```
Step 1: Mean = 67.2
Step 2: Find squared deviations from mean for each value, sum them
Step 3: Variance = Σ(x - mean)² / n
Step 4: Standard Deviation = √Variance
```

Calculating: $\Sigma(x-67.2)^2 \approx 1666.0$ (approx)

```
Variance = 1666.0 / 10 = 166.6
Standard Deviation = √166.6 ≈ 12.91
```

Represents: How spread out the data values are from the mean. A low SD means data points are close to the mean; a high SD means data is widely dispersed.

Python Verification:

```python
        import numpy as np
```

```
data = [45,55,60,60,65,70,72,75,80,90]
print("Mean:", np.mean(data))
print("Median:", np.median(data))
from scipy import stats
print("Mode:", stats.mode(data, keepdims=True))
print("Std Dev:", np.std(data))
```

Summary of what each measures: - **Mean** → Central tendency (average) - **Median** → Central tendency (positional, robust to outliers) - **Mode** → Most common/frequent value - **Standard Deviation** → Variability/dispersion of data around the mean

---

Q17. Explain how to identify and handle outliers in a dataset using suitable techniques such as filtering and discretization. (10M)

Definition of Outliers: Outliers are data points that significantly differ from other observations in the dataset — either unusually high or unusually low — which can distort statistical analyses and machine learning models.

Techniques to Identify Outliers:

1. Visualization Methods - **Box Plot:** Points outside the “whiskers” (beyond 1.5×IQR) are considered outliers. - **Scatter Plot:** Visually identify points far from the general cluster. - **Histogram:** Identify unusually spaced bars/gaps.

2. Statistical Methods

a) Z-Score Method

```
Z = (X - μ) / σ
```

A data point with |Z| > 3 is typically considered an outlier.

b) IQR (Interquartile Range) Method

```
IQR = Q3 - Q1
Lower Bound = Q1 - 1.5 × IQR
Upper Bound = Q3 + 1.5 × IQR
```

Any value below the Lower Bound or above the Upper Bound is an outlier.

Example: Data = [10, 12, 12, 13, 12, 11, 14, 13, 15, 100] Here, 100 is clearly an

outlier compared to the rest of the values.

```
import numpy as np
data = [10,12,12,13,12,11,14,13,15,100]
Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
outliers = [x for x in data if x < lower or x > upper]
print("Outliers:", outliers)
```

Techniques to Handle Outliers:

1. Filtering (Removal) - Remove rows containing outlier values if they are due to data entry errors.

```
df = df[(df['col'] >= lower) & (df['col'] <= upper)]
```

2. Capping/Winsorization - Replace outliers with the nearest acceptable boundary value (upper/lower bound) instead of removing them.

```
df['col'] = np.where(df['col'] > upper, upper, df['col'])
df['col'] = np.where(df['col'] < lower, lower, df['col'])
```

3. Transformation - Apply log, square root, or Box-Cox transformation to reduce the impact/skewness caused by outliers.

4. Discretization (Binning) - Convert continuous data into discrete bins/categories, which reduces the effect of extreme individual values since they get grouped into a range.

```
df['col_binned'] = pd.cut(df['col'], bins=[0,20,40,60,80,100],
                          labels=['Very Low','Low','Medium','High','Very High
```

This way, an extreme outlier simply falls into the “Very High” bin rather than skewing numeric computations.

5. Imputation - Replace outlier values with mean/median, similar to missing value treatment.

Conclusion: The choice of technique (filtering, capping, transformation, or discretization) depends on whether the outlier represents a genuine rare event

(which may need to be kept, e.g., fraud detection) or a data error (which should be corrected/removed).

End of Answer Bank – Unit I & Unit II (Data Science)